

K8S-08: DQL Queries for Kubernetes

Series: K8S — Kubernetes Monitoring | **Notebook:** 8 of 13 | **Created:** January 2026 | **Last Updated:** 09/25/2026

Advanced Query Patterns for Kubernetes Data

This notebook provides a comprehensive reference of DQL queries for Kubernetes monitoring. From basic entity queries to complex performance analysis, these patterns help you extract insights from your Kubernetes data.

Table of Contents

- 1. [Entity Queries](#)
- 2. [Metric Queries](#)
- 3. [Log and Event Queries](#)
- 4. [Trace Queries](#)
- 5. [Correlation Queries](#)
- 6. [Dashboard Queries](#)
- 7. [Alerting Queries](#)
- 8. [Query Templates](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Kubernetes monitoring
Permissions	<code>metrics.read</code> , <code>entities.read</code> , <code>logs.read</code> , <code>events.read</code>

Requirement	Details
Knowledge	K8S-01 through K8S-07
Data	Active Kubernetes cluster monitored

1. Entity Queries

Kubernetes Entity Types

Entity Type	DQL Table	Description
Cluster	dt.entity.kubernetes_cluster	K8s clusters
Node	dt.entity.kubernetes_node	Worker nodes
Namespace	dt.entity.cloud_application_namespace	Namespaces
Workload	dt.entity.cloud_application	Deployments, StatefulSets
Container	dt.entity.container_group_instance	Container instances
Service	dt.entity.service	Detected services

```
// List all Kubernetes clusters (smartscape topology)
smartscapeNodes "K8S_CLUSTER"
| fields entity.name = name, tags
| sort entity.name asc

// Legacy alternative (deprecated for new content):
// fetch dt.entity.kubernetes_cluster
// | fields entity.name, tags
// | sort entity.name asc
```

```
// List all Kubernetes nodes (smartscape topology)
smartscapeNodes "K8S_NODE"
| fields entity.name = name, tags
| sort entity.name asc

// Legacy alternative (deprecated for new content):
// fetch dt.entity.kubernetes_node
// | fields entity.name, tags
// | sort entity.name asc
```

```
// List all namespaces (smartscape topology)
smartscapeNodes "K8S_NAMESPACE"
| fields entity.name = name, tags
| sort entity.name asc

// Legacy alternative (deprecated for new content):
// fetch dt.entity.cloud_application_namespace
// | fields entity.name, tags
// | sort entity.name asc
```

```
// List all workloads – Deployments (smartscape topology)
smartscapeNodes "K8S_DEPLOYMENT"
| fields entity.name = name, tags
| sort entity.name asc
| limit 50

// For other workload types, substitute:
// smartscapeNodes "K8S_STATEFULSET"
// smartscapeNodes "K8S_DAEMONSET"

// Legacy alternative (deprecated for new content):
// fetch dt.entity.cloud_application
// | fields entity.name, tags
// | sort entity.name asc
// | limit 50
```

```
// Count Kubernetes nodes (smartscape topology)
smartscapeNodes "K8S_NODE"
| summarize nodeCount = count()
```

2. Metric Queries

Common Kubernetes Metrics

The table below lists keys verified present on a live tenant (yhu28601 , 08/11/2026). Enumerate your own before writing a query — this is what exists there, not a guarantee for every estate:

```
metrics
| filter startsWith(metric.key, "dt.kubernetes")
| summarize n = count(), by:{metric.key}
| sort metric.key asc
```

Two mechanics that bite here. `metrics` takes `from:` with no leading comma — `metrics from:-2h | ...`. Writing `metrics, from:-2h` is a `PARSE_ERROR`, and a parse error read only for its record count is indistinguishable from "this tenant carries no such metrics." And `metrics` returns one row per dimension tuple, not one per key, so `summarize by:{metric.key}` is what collapses the result into a key list.

Metric	Description	Unit	Grain
<code>dt.kubernetes.container.cpu_usage</code>	Container CPU usage	Millicores	container
<code>dt.kubernetes.container.memory_working_set</code>	Container memory (active)	Bytes	container
<code>dt.kubernetes.container.requests_cpu</code>	CPU requests	Millicores	container
<code>dt.kubernetes.container.requests_memory</code>	Memory requests	Bytes	container
<code>dt.kubernetes.container.limits_cpu</code>	CPU limits	Millicores	container
<code>dt.kubernetes.container.limits_memory</code>	Memory limits	Bytes	container
<code>dt.kubernetes.container.cpu_throttled</code>	CPU throttling	—	container

Metric	Description	Unit	Grain
<code>dt.kubernetes.container.restarts</code>	Restart count (delta)	Count	container
<code>dt.kubernetes.container.oom_kills</code>	OOMKilled count	Count	container
<code>dt.containers.cpu.throttled_time</code>	CPU throttle time	Nanoseconds	container
<code>dt.kubernetes.node.cpu_allocatable</code>	Node allocatable CPU	Millicores	node
<code>dt.kubernetes.node.memory_allocatable</code>	Node allocatable memory	Bytes	node
<code>dt.kubernetes.node.pods_allocatable</code>	Node allocatable pod slots	Count	node
<code>dt.kubernetes.node.conditions</code>	Node condition status	—	node
<code>dt.kubernetes.pod.containers_desired</code>	Desired containers per pod	Count	pod
<code>dt.kubernetes.pod.network_received_data / .network_transmitted_data</code>	Pod network throughput	Bytes	pod
<code>dt.kubernetes.workload.pods_desired</code>	Desired pods per workload	Count	workload
<code>dt.kubernetes.workload.conditions</code>	Workload condition status	—	workload
<code>dt.kubernetes.nodes / .pods / .containers / .workloads / .events</code>	Object counts	Count	cluster

There is no node- or workload-grain *usage* or *requests* metric.

`dt.kubernetes.node.cpu_usage` , `dt.kubernetes.node.memory_usage` ,
`dt.kubernetes.workload.requests_cpu` and
`dt.kubernetes.workload.requests_memory` **do not exist** — each returns zero rows from `metrics | filter metric.key == "..."` (checked 08/11/2026). Node and workload views are **derived**: sum the `dt.kubernetes.container.*` series across `k8s.node.name` or `k8s.workload.name` . Only the `*_allocatable` family is genuinely node-grain.

This is worth stating plainly because a `timeseries` against a key that does not exist is syntactically valid, executes without error, and returns nothing — the same shape as a correct query against an idle cluster. §5 below shows the derivation for nodes.

Reading request and limit metrics across ActiveGate upgrades. The CPU and memory *request* and *limit* metrics carry an **effective pod value**. Where a simple sum of container values would not match what Kubernetes enforces, Dynatrace adds a **compensating delta data point** to the same metric at ingest — in the docs' words, *"Dynatrace calculates the correct effective value at ingest time and adjusts the metric so that aggregated views across pods, workloads, namespaces, and clusters always reflect what Kubernetes actually enforces."* What that effective value covers grew in ActiveGate steps:

ActiveGate	What the request/limit metrics account for
1.339+	App containers and native sidecars (<code>restartPolicy: Always</code>)
1.343+	Init containers — a delta where the init phase peaks above the running-phase sum
1.345+ (rollout from 08/25/2026)	Pod-level <code>spec.resources</code> (Kubernetes 1.34+) and <code>RuntimeClass</code> overhead — including a negative delta where a pod-level <i>limit</i> caps the total below the container sum
1.347 (pre-release; rollout planned from 09/22/2026)	Fix for a miscalculated effective value when a native sidecar precedes one or more non-restartable init containers

Metric family	Affected by the effective-resource changes?
<code>dt.kubernetes.container.requests_cpu</code> , <code>.requests_memory</code> , <code>.limits_cpu</code> , <code>.limits_memory</code>	Yes — the effective value changes at each boundary above

Metric family	Affected by the effective-resource changes?
<code>dt.kubernetes.container.cpu_usage</code> , <code>.memory_working_set</code>	No — usage metrics are unchanged

Three practical consequences:

1. **A trend that spans any of these ActiveGate versions may carry a discontinuity** — up for init containers, up **or down** for pod-level limits, with no workload change at all. It appears only where the pod shape triggers it (an init container larger than the running-phase sum, a pod-level `spec.resources` , or an `overhead`), so it is not a uniform shift. Do not read a request/limit chart across a boundary as a single trend, and do not let a threshold alert on reserved CPU or memory fire on the discontinuity. Bound your comparison windows on one side of the upgrade or the other.
2. **Earlier baselines misstate reservations** relative to what Kubernetes enforces. The post-upgrade figures are the more faithful number — re-baseline rather than trying to reconcile the two.
3. **The delta belongs to no container.** The compensating data point has no `dt.smartscape.container` dimension (its `dt.smartscape_source.type` is `K8S_POD`), so a breakdown *by container* shows it as an unnamed row. Summing across it gives the enforced pod value; filtering out rows without a container quietly brings back the old arithmetic.

Because node and namespace roll-ups are *derived* from these container-grain series (see the note above), a change propagates to every level you aggregate to — there is no unaffected roll-up to fall back on. One edge case the docs call out explicitly: *"Upgrading Istio to version 1.27+ (where native sidecars are the default) without also upgrading to ActiveGate version 1.339+ will cause sidecar resources to be missing from the metric."*

Each version rolls out per ActiveGate, not per tenant, and ActiveGate fleets lag tenant version — so **verify the version of the ActiveGate carrying the `kubernetes-monitoring` capability for the cluster in question** before deciding which side of a boundary a data point sits on. Until a given version reaches that ActiveGate, the earlier reading is what your data shows, and everything above about baselines and alert thresholds still describes it correctly.

Sources: [Effective pod resources \(DT docs\)](#), [ActiveGate 1.345 release notes \(DT docs\)](#), [ActiveGate 1.347 release notes \(DT docs\)](#) — read 09/18/2026.

```
// Container CPU usage - top consumers
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage), from:-1h,
by:{dt.entity.container_group_instance}
| sort avgCpuMillicores desc
| limit 15
```

```
// Container memory usage approaching limits
timeseries avgMemBytes = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{dt.entity.container_group_instance}
| fieldsAdd avgMemBytesValue = arrayAvg(avgMemBytes)
| sort avgMemBytesValue desc
```

```
// CPU throttling detection
timeseries totalThrottle = sum(dt.containers.cpu.throttled_time), from:-1h, by:
{dt.entity.container_group_instance}
| fieldsAdd totalThrottleValue = arrayAvg(totalThrottle)
| filter totalThrottleValue > 0
| sort totalThrottleValue desc
| limit 15
```

```
// Namespace-level resource usage – sum() of every container in the namespace
// rollup: avg averages each container within a time bucket before the cross-
container sum;
// without it, sum() also adds the 1-min points in each bucket and wide
timeframes read ~10x high.
timeseries cpuMillicores = sum(dt.kubernetes.container.cpu_usage, rollup: avg),
from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgCpuMillicores = arrayAvg(cpuMillicores)
| sort avgCpuMillicores desc
| limit 15
```

```
// Resource requests by namespace (capacity planning)
// Requests are container-grain; sum them to reach the namespace figure.
// rollup: avg averages each container within a time bucket before the cross-
container sum;
// without it, sum() also adds the 1-min points in each bucket and wide
timeframes read ~10x high.
timeseries cpuReq = sum(dt.kubernetes.container.requests_cpu, rollup: avg),
from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgReqMillicores = round(arrayAvg(cpuReq), decimals: 0)
| fields k8s.namespace.name, avgReqMillicores
| sort avgReqMillicores desc
| limit 15
```


3. Log and Event Queries

Logs and Kubernetes events are two different objects, and the split is not optional.

Application output written to stdout/stderr lands in `logs`. Kubernetes events —

`FailedScheduling`, `BackOff`, `Unhealthy`, `Killing` — land in `events`, and are **not** searchable through `logs`.

Getting this wrong is expensive in both directions:

Anti-pattern	Result on a live tenant (08/11/2026)
<pre>fetch logs \ filter matchesPhrase(log.source, "kubernetes")</pre>	0 rows after scanning 46.5 GB. Empty <i>and</i> billed.
<pre>fetch events \ filter event.kind == "K8S_EVENT"</pre>	0 rows. <code>event.kind</code> never takes that value — over 24 h the only values present were <code>DAVIS_EVENT</code> , <code>SYNTHETIC_EVENT</code> , <code>FLEET_EVENT</code> , <code>DAVIS_PROBLEM</code> . Kubernetes events carry <code>DAVIS_EVENT</code> , so <i>kind</i> cannot discriminate them.
<pre>k8s.event.reason</pre>	Field resolves, null on every record.

The working pair is `event.provider == "KUBERNETES_EVENT"` plus

`dt.kubernetes.event.reason`. Companion fields: `dt.kubernetes.event.message`, `.count`, `.first_seen`, `.last_seen`, `.involved_object.kind`, `.involved_object.name`, alongside `k8s.cluster.name` / `k8s.namespace.name` / `k8s.pod.name` / `k8s.workload.name` / `k8s.node.name`.

Log Query Patterns

```
// Error logs by namespace
fetch logs, from:-1h
| filter loglevel == "ERROR"
| filter isNotNull(k8s.namespace.name)
| summarize errorCount = count(), by:{k8s.namespace.name}
| sort errorCount desc
| limit 15
```

```
// Kubernetes events – what your estate actually emits, by reason
fetch events, from:-1h
| filter event.provider == "KUBERNETES_EVENT"
| summarize eventCount = count(), by:{dt.kubernetes.event.reason}
| sort eventCount desc
| limit 30
```

```
// Pod crash / restart loops – the BackOff event carries the pod and the reason
fetch events, from:-1h
| filter event.provider == "KUBERNETES_EVENT"
| filter dt.kubernetes.event.reason == "BackOff"
| fields timestamp, k8s.cluster.name, k8s.namespace.name,
      dt.kubernetes.event.involved_object.name,
      dt.kubernetes.event.message
| sort timestamp desc
| limit 20
```

```
// OOM kills – a metric, not an event
// Dynatrace derives this counter from the container status and writes it only
// when at least one kill occurred, so any non-null value is a real OOM.
timeseries oom = sum(dt.kubernetes.container.oom_kills), from:-24h,
  by:{k8s.cluster.name, k8s.namespace.name, k8s.workload.name}
| fieldsAdd oomTotal = arraySum(oom)
| filter oomTotal > 0
| fields k8s.cluster.name, k8s.namespace.name, k8s.workload.name, oomTotal
| sort oomTotal desc
| limit 20
```

```
// Log volume trend by namespace
fetch logs, from: now() - 24h
| filter isNotNull(k8s.namespace.name)
| summarize log_count = count(), by:{k8s.namespace.name, time_bucket =
bin(timestamp, 1h)}
| sort time_bucket asc
| limit 200
```

4. Trace Queries

Span Queries for Kubernetes Services

`duration` is a duration type — compare it against a duration literal (`1s` , `500ms`), never a nanosecond integer. `duration > 10000000000` is a type mismatch: it raises only a `DATATYPE_MISMATCH` warning, evaluates to null, and the filter silently drops every span. Spans also have no `timestamp` field — their time fields are `start_time` and `end_time`.

```
// Service response times in K8s
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(k8s.namespace.name)
| summarize
    p50 = percentile(duration, 50),
    p95 = percentile(duration, 95),
    p99 = percentile(duration, 99),
    by:{k8s.namespace.name, dt.entity.service}
| sort p99 desc
| limit 15
```

```
// Error rate by service
// Three things to get right here, each of which fails silently:
// 1. The field is span.status_code (stable). otel.status_code does not exist.
// 2. Values are LOWERCASE – "error", not "ERROR".
// 3. span.status_code is NULL on almost every successful span, so derive
//    successes as total - errors rather than counting a non-error value.
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(k8s.namespace.name)
| summarize {
    total = count(),
    errors = countIf(span.status_code == "error")
}, by:{k8s.namespace.name, dt.entity.service}
| fieldsAdd successes = total - errors
| fieldsAdd errorRate = round(100.0 * errors / total, decimals: 2)
| filter errors > 0
| sort errorRate desc
| limit 15
```

```
// Slow traces (>1 second)
fetch spans, from:-1h
| filter span.kind == "server" and duration > 1s
| filter isNotNull(k8s.namespace.name)
| fields start_time, trace.id, k8s.namespace.name, span.name, duration
| sort duration desc
| limit 20
```

```
// Request throughput by namespace
fetch spans, from:-1h
| filter span.kind == "server"
| filter isNotNull(k8s.namespace.name)
| summarize requestCount = count(), by:{k8s.namespace.name}
| sort requestCount desc
| limit 10
```

5. Correlation Queries

Joining Multiple Data Sources

```
// High CPU workloads with their names
timeseries avgCpuMillicores = avg(dt.kubernetes.container.cpu_usage), from:-1h,
by:{dt.entity.cloud_application}
| fieldsAdd avgCpuMillicoresValue = arrayAvg(avgCpuMillicores)
| lookup [fetch dt.entity.cloud_application | fields id, entity.name],
sourceField:dt.entity.cloud_application, lookupField:id
| fieldsRename workloadName = lookup.entity.name
| fields workloadName, avgCpuMillicoresValue
| sort avgCpuMillicoresValue desc
```

```
// Nodes with high utilization – derived, because no node-grain usage metric
exists
// used = container CPU summed across every container on the node
// alloc = the one genuinely node-grain family
// rollup: avg averages each container within a time bucket before the cross-
container sum;
// without it, sum() also adds the 1-min points in each bucket and wide
timeframes read ~10x high.
timeseries {
    used = sum(dt.kubernetes.container.cpu_usage, rollup: avg),
    allocatable = avg(dt.kubernetes.node.cpu_allocatable)
}, from:-1h, by:{k8s.cluster.name, k8s.node.name}
| fieldsAdd cpuPercent = round(100 * arrayAvg(used) / arrayAvg(allocatable),
decimals: 1)
| filter cpuPercent > 70
| fields k8s.cluster.name, k8s.node.name, cpuPercent
| sort cpuPercent desc
```

6. Dashboard Queries

Queries Optimized for Dashboards

```
// Cluster summary tile (smartscape topology)
smartscapeNodes "K8S_CLUSTER"
| summarize clusterCount = count()
```

```
// Node count tile (smartscape topology)
smartscapeNodes "K8S_NODE"
| summarize nodeCount = count()
```

```
// Error rate trend (chart)
fetch logs, from: now() - 24h
| filter loglevel == "ERROR" and isNotNull(k8s.namespace.name)
| summarize errors = count(), by:{time_bucket = bin(timestamp, 1h)}
| sort time_bucket asc
```

```
// Top namespaces by CPU (bar chart) – sum() of every container in the namespace
// rollup: avg averages each container within a time bucket before the cross-
// container sum;
// without it, sum() also adds the 1-min points in each bucket and wide
// timeframes read ~10x high.
timeseries cpuMillicores = sum(dt.kubernetes.container.cpu_usage, rollup: avg),
from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgCpuMillicores = arrayAvg(cpuMillicores)
| sort avgCpuMillicores desc
| limit 10
```

7. Alerting Queries

Threshold-Based Alerting Patterns

```
// High memory usage alert query
timeseries avgMemBytes = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{dt.entity.container_group_instance}
| fieldsAdd avgMemBytesValue = arrayAvg(avgMemBytes)
```

```
// CrashLoop detection (last hour) – alerting condition
// The event reason is "BackOff"; "CrashLoopBackOff" appears in the message
// text,
// so filter on the reason and read the message for detail.
fetch events, from:-1h
| filter event.provider == "KUBERNETES_EVENT"
| filter dt.kubernetes.event.reason == "BackOff"
| summarize crashLoopEvents = count(),
    affectedPods = countDistinctExact(dt.kubernetes.event.involved_object.name)
```

```
// Error rate spike detection
fetch logs, from: now() - 1h
| filter loglevel == "ERROR" and isNotNull(k8s.namespace.name)
| summarize errors = count(), by:{k8s.namespace.name}
| filter errors > 100
| sort errors desc
```

8. Query Templates

Reusable Query Patterns

Filter by Namespace:

```
| filter k8s.namespace.name == "NAMESPACE_NAME"
```

Filter by Cluster:

```
| filter k8s.cluster.name == "CLUSTER_NAME"
```

Time-based Analysis:

```
fetch logs, from:-DURATION  
| summarize log_count = count(), by:{time_bucket = bin(timestamp, INTERVAL)}  
| sort time_bucket asc
```

Top N Pattern:

```
| sort METRIC desc  
| limit N
```

Summary

This notebook provided DQL query patterns for:

- Entity discovery and relationships
 - Container and node metrics
 - Log and event analysis
 - Distributed tracing
 - Multi-source correlation
 - Dashboard visualizations
 - Alerting conditions
-

References

- [Dynatrace Query Language \(DQL\) reference \(DT docs\)](#)
 - [smartscapeNodes command \(DT docs\)](#)
 - [Set up Dynatrace on Kubernetes \(DT docs\)](#)
 - [Kubernetes app — workloads + namespaces views \(DT docs\)](#)
 - [Davis Problems app \(DT docs\)](#)
 - [Effective pod resources \(DT docs\)](#)
 - [ActiveGate 1.345 release notes \(DT docs\)](#)
 - [ActiveGate 1.347 release notes \(DT docs\)](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.